Proiect de atestat profesional la Informatică

LED-Dress

Profesor coordonator:

Florea Delilah

Numele și prenumele candidatului:

Florea Deborah Ruxandra Maria

Sibiu, 2023

1. Introducere

Acestă lucrare reprezintă un proiect wearable IoT, respectiv o rochie luminată de 300 de LED-uri conectate la un microcontroller. LED-urile sunt controlate de către o aplicaţie Android, care transmite date microcontroller-ului, precum culoare, număr de LED-uri aprinse şi algoritmul folosit pentru luminarea acestora.

Proiecte asemănătoare s-au mai regăsit la lucrări de licenţă dar folosind alte componente hardware. De acolo a apărut şi intenţia de dezvoltare a proiectului, care a întâlnit dorinţa de a avea o rochie memorabilă şi unică la banchetul de clasa a 12-a.

Fig. 1: Rochia luminând folosind algoritmul Gradient

2. Descrierea aplicaţiei

Aplicația Android a fost dezvoltată în limbajul Kotlin în mediul de programare Android Studio, iar modul de luminare al LED-urilor a fost programat în C++ în mediul de programare Arduino IDE. Pe partea de hardware am folosit un microcontroller ESP32 cu Wi-Fi incorporat, care a ajutat la transmiterea de date între telefon şi plăcuţă. Cele 300 de LED-uri se regăsesc pe o bandă de 5m (câte 60 LED-uri pe metru), care a fost cusută pe o crinolină. Acestea sunt conectate prin cabluri mamă-tată la microcontroller-ul ESP32, care este apoi alimentat cu o baterie externă printr-un cablu USB.

Prin intermediul componentei Wi-Fi incorporate telefonul cu sistem Android se conectează la microcontroller şi transmite din aplicaţie date conform cărora LED-urile se vor aprinde în culoarea aleasă şi în diverse modele.

Fig. 2 & 3: Rochia luminând folosind algoritmul Amplitude

3. Interfaţa

Interfaţa aplicaţiei Android prezintă în partea de sus a ecranului amplitudinea receptată de microfon în funcţie de care sunt calculate mai apoi numărul de LED-uri care ar trebui aprinse. Pe ecran se regăseşte de asemenea un component Color Picker care permite utilizatorului să selecteze culoarea şi intensitatea acesteia, pe care doreşte să o afişeze LED-urile. De asemenea, în aplicaţie se regăsesc şi butoane care dau utilizatorului posibilitatea de a alege modelul reprezentat de banda de LED-uri (ex.: în funcţie de amplitudine, o singură culoare, curcubeu progresiv ş.a.).

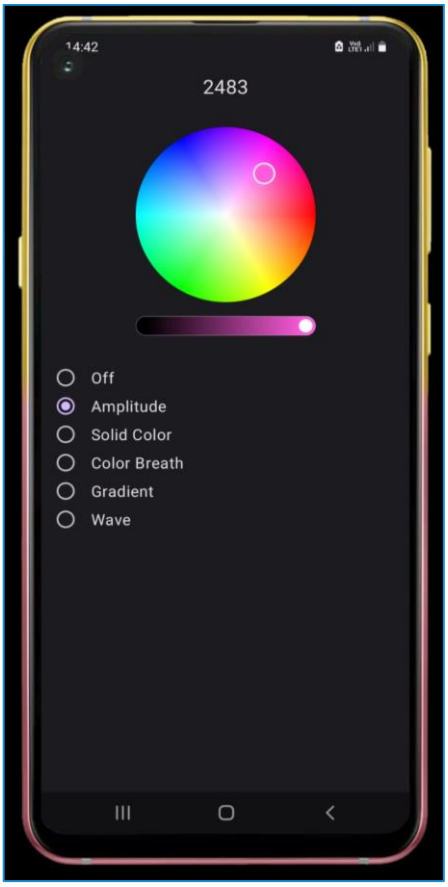

Fig. 4: Interfața aplicației

4. Conectarea prin Wi-Fi

Comunicarea dintre microcontroller şi aplicaţie se săvârşeşte prin intermediul unui UDP Server (User Datagram Protocol Server). Acesta presupune transmiterea de pachete între ESP32 şi smartphone utilizând adresa IP adresată plăcuţei şi portul utilizat de aceasta. Aici este de folos componenta de Wi-Fi integrată în ESP32, deoarece ne oferă posibilitatea de a configura microcontroller-ul ca un Acess Point, ceea ce presupune că alte dispozitive cu capabilități Wi-Fi se vor putea conecta la rețeaua creată de acesta și vor putea comunica fără a avea nevoie de un router. A fost folosit acest tip de server pentru eficientizarea trimiterii datelor, mai exact protocolul UDP este un protocol “fără conexiune” (connectionless), adică nu presupune verificarea unei conexiuni între cei doi clienți înainte de trimiterea pachetelor. Astfel, se salvează timp dar se poate risca primirea defectuoasă a pachetelor. Totuși am optat pentru acest tip de server, deoarece dorim să schimbăm numărul de LED-uri aprinse la fiecare amplitudine receptată de către microfonul telefonului. De asemenea, ESP32 permite folosirea unor librării ”asincrone”, care funcționează în fundal, reducând nevoia de a implementa funcționalități pentru primiterea sau procesarea datelor.

```
Serial.begin(115200);
WiFi.mode(WIFI_AP);
WiFi.softAP(ssid, password);

if (udp.listen(1234)) {
  Serial.print("UDP Listening on IP: ");
  Serial.println(WiFi.localIP());
  udp.onPacket([](AsyncUDPPacket& packet) {
    // Parse and convert the received bytes:
    memcpy(&data, packet.data(), packet.length());
    data.amplitude = swap_endian(data.amplitude);
```

Fig. 5: Arduino IDE

```
class UdpServer(serverPort: Int) {

    private val socket = DatagramSocket(serverPort)
    private val address: InetAddress =
        InetAddress.getByName( host: "192.168.4.1")

    Razvan Filea +1
    fun send(color: Color, amplitude: Int, algorithm: LedAlgorithm) {
        val buffer = ByteBuffer.allocate( capacity: Int.SIZE_BYTES + Byte.SIZE_BYTES * 4).apply {
            putInt(amplitude)
            put(color.red().toByte())
            put(color.green().toByte())
            put(color.blue().toByte())
            put(algorithm.toByte())
        }
```

Fig. 6: Android Studio

5. Programarea LED-urilor

LED-urile au fost programate folosind librăria FastLED, o librărie eficientă și rapidă pentru programarea benzilor de LED-uri adresabile, adică LED-uri care au cipuri unice, permițând astfel controlarea individuală a acestora. Astfel, definim un vector CRGB pentru cele 300 de LED-uri pe care le avem pentru a le putea manipula mai apoi folosind librăria FastLED.

```
FastLED.addLeds<WS2812, LED_PIN, GRB>(led, NUM_LEDS);
pinMode(LED_PIN, OUTPUT);
digitalWrite(LED_PIN, HIGH);
```

Fig. 7: Configurare

```
for (int i = 0; i < NUM_LEDS; i++)
  led[i] = CRGB(0,0,0);
FastLED.show();
```

Fig. 8: Setare culoare și afișare

6. Receptarea sunetului

Pentru receptarea sunetului am folosit clasa MediaRecorder din Kotlin, folosită adesea pentru înregistrarea fișierelor audio sau video. Clasa permite selectarea sursei de preluare a sunetului, în cazul nostru microfonul telefonului, și setarea tipului de fișier și locația în care să salveze datele preluate.

```
private val recorder = MediaRecorder()

Razvan Filea
fun start(callback: (amplitude: Int) -> Unit) {
    running = true
    thread {
        recorder.setAudioSource(MediaRecorder.AudioSource.MIC)
        recorder.setOutputFormat(MediaRecorder.OutputFormat.THREE_GPP)
        recorder.setAudioEncoder(MediaRecorder.AudioEncoder.AMR_NB)
        recorder.setOutputFile(File(ctx.cacheDir, child: "temp.wav"))
        recorder.prepare()
        recorder.start()

        while (running) {
            Thread.sleep( millis: 400)
            val maxAmplitude = recorder.maxAmplitude

            Log.i( tag: "Amplitude", maxAmplitude.toString())
            callback(maxAmplitude)
        }

        recorder.stop()
        recorder.reset()
        recorder.release()
    }
}
```

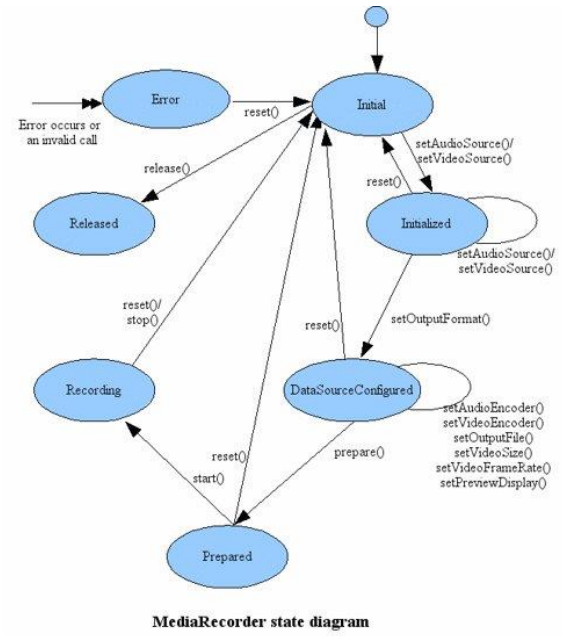

Fig. 9 & 10: Utilizarea variabilei MediaRecorder & modul acesteia de funcționare explicat printr-o diagramă

Deoarece valoarea maximă a amplitudinii receptate este 32767 (maximum short) am decis să împărțim valoarea primită la maximul posibil pentru a obține un procentaj. Acesta este mai apoi înmulțit cu 300, respectiv numărul nostru de LED-uri și se află astfel un întreg care reprezintă numărul de LED-uri ce urmează a fi aprinse.

```
const float amplitude_percetage = float(data_cpy.amplitude) / SHRT_MAX;
const int num_leds = NUM_LEDS * amplitude_percetage;
```

Fig. 11: Calculare număr LED-uri

7. Algoritmi folosiţi

În funcție de algoritmul selectat LED-urile se vor aprinde un număr anume în funcție de amplitudinea receptată (AMPLITUDE), toate într-o singură culoare (SOLID_COLOR), într-un curcubeu mișcător (GRADIENT), într-o prezentare progresivă (WAVE), sau vor rămâne stinse (OFF). Acești algoritmi sunt stocați într-o clasă enum care ne permite să adăugăm sau să ștergem dintre aceștia mult mai ușor, transformând mai apoi obiectele în bytes pentru a-i putea trimite prin serverul UDP.

```
fun toByte(): Byte {
    return when (this) {
        OFF -> 0
        AMPLITUDE -> 1
        SOLID_COLOR -> 2
        COLOR_BREATH -> 3
        GRADIENT -> 4
        WAVE -> 5
    }
}
```

Fig. 12: Transformarea obiectelor din clasa enum în bytes

8. Rularea programului

Deoarece poate exista riscul ca telefonul să se stingă sau ca aplicația să fie închisă am decis să rulăm serviciul acesteia în fundal. Astfel, oricând se va distruge activitatea aplicației, programul va continua să ruleze în fundal, iar utilizatorul va fi alertat de acest lucru printr-o notificare care va afișa algoritmul folosit și culoarea aleasă. Dacă utilizatorul va redeschide aplicația, aceasta se va reconstitui utilizând datele procesate în continuare în fundal.

Fig. 13: Notificarea trimisă de aplicație

```
private fun sendStatus() {
    val statusIntent = Intent().apply {
        action = STATUS_ACTION
        putExtra(COLOR_KEY, color.toArgb())
        putExtra(AMPLITUDE_KEY, amplitude)
        putExtra(ALGORITHM_KEY, ledAlgorithm.toByte())
    }

    sendBroadcast(statusIntent)
}

Razvan Filea +1
private fun sendAmplitude() {
    val statusIntent = Intent().apply {
        action = STATUS_ACTION
        putExtra(AMPLITUDE_KEY, amplitude)
    }

    sendBroadcast(statusIntent)
}
```

Fig. 14: Transmiterea din serviciu către activitate

```
val statusFilter = IntentFilter()
statusFilter.addAction(NotificationService.STATUS_ACTION)
statusReceiver = object : BroadcastReceiver() {
    override fun onReceive(p0: Context?, intent: Intent) {
        // Color is optional
        val newColor = intent.getIntExtra(NotificationService.COLOR_KEY,  defaultValue: -1)
        if (newColor != -1) {
            selectedColor = Color(newColor)
        }
        // Algorithm is optional
        val newAlgorithm =
            intent.getByteExtra(NotificationService.ALGORITHM_KEY, 255.toByte())
        if (newAlgorithm != 255.toByte()) {
            selectedAlgorithm = LedAlgorithm.fromByte(newAlgorithm)
        }

        amplitude = intent.getIntExtra(NotificationService.AMPLITUDE_KEY,  defaultValue: 0)
    }
}
registerReceiver(statusReceiver, statusFilter)
```

Fig. 15: Receptarea datelor de către activitate

9. Bibliografie

Lucrare de disertație: ”Tehnologie cu amprentă feminină”, Ifrim Silvia-Georgiana, Sibiu 2020, ULBS, Facultatea de Inginerie, Departamentul de Calculatoare și Inginerie Electrică

https://developer.android.com/reference/android/media/MediaRecorder

https://stackoverflow.com/questions/54260554/how-to-use-android-media-recorder-to-get-max-amplitude

https://developer.android.com/reference/java/net/DatagramSocket

https://www.instructables.com/ESP32-Web-Server-Controlling-LED-Lights/

https://randomnerdtutorials.com/esp32-useful-wi-fi-functions-arduino/

https://developer.android.com/guide/components/foreground-services#handle-user-initiated-stop

https://blog.claytonk.com/2019/10/16/working-with-fastled-library/

https://fastled.io/docs/3.1/struct_c_r_g_b.html#ac97afb676b3be14769ede58a55b1078a

https://betterprogramming.pub/implementing-a-foreground-service-in-android-building-a-stopwatch-app-5a1da85d929f